



FAST NUCES

DEPARTMENT OF MANAGEMENT

Course Progress Reflection Report

Programming for Business Project

Student Name: Noor ul Huda

Roll No: 24i-5657

Teacher: Faheem ur Rehman

Degree: BS – FinTech

University: FAST NUCES

16 May 2025

Contents

1	Introduction	3
1.1	What is Programming?	3
1.2	Why Do We Use Flowcharts?	3
2	Basic Flowchart	3
3	Primitive Data Types and Input/Output	4
3.1	Primitive Data Types Defined	4
3.2	Input and Output	4
3.3	Signed vs. Unsigned Data	5
3.4	Constants and Variables	5
4	Operators	5
4.1	Arithmetic Operators	5
4.2	Increment and Decrement Operators	5
4.3	Relational Operators	5
4.4	Logical Operators	6
4.5	Type Casting	6
5	Bitwise Operators	6
6	Conditional/Selection Structures	6
6.1	If and If-Else	6
6.2	Nested If-Else	6
6.3	Else-If Ladder	6
6.4	Switch Statement	6
6.5	Ternary Operator	7
7	Loops	7
7.1	While Loop	7
7.2	For Loop	7
7.3	Do-While Loop	7
7.4	Nested Loops	7
7.5	Break and Continue	8
8	Functions	8
8.1	Function Prototypes	8
8.2	Definition and Calling	8
8.3	Function Overloading	8
8.4	Default Parameters	8
9	Arrays	8
9.1	Initializing Arrays	9
9.2	Character Arrays	9
10	Conclusion	9

List of Figures

1	Basic flowchart representation [1]	4
2	Flowchart structure and logic flow [3]	5
3	For loop illustration [2]	7
4	Array structure and representation [4]	9

1 Introduction

1.1 What is Programming?

Programming refers to the art and science of making a computer execute certain jobs or solve some issues by authoring a series of instructions using a language understood by the computer. These instructions are referred to as code, and programming languages such as C++, Python, and Java aid us in coding. Coding is an essential skill in the modern computer age since computers find application in nearly all industries, ranging from business and medicine to learning and leisure. Knowing how to code enables one to develop software, parse data, automate processes, and even design intelligent systems.

Additionally, programming is not merely codewriting; it promotes critical thinking and problem-solving. Learning programming, you learn to analyze problems, think critically, and apply good solutions. These intellectual skills are easily applied across different fields outside technology, making programming such an invaluable and transferable skill.

1.2 Why Do We Use Flowcharts?

Flowcharts represent the logical operations of a program graphically. They are a big help while planning software development. By producing a flowchart, programmers are able to graph the whole program logic before programming even begins. This helps a programmer catch bugs or inefficiency earlier and get an idea about the workflow as well as understand how various program segments interact.

Flowcharts have definite symbols: ovals denote start and end, rectangles indicate action or process, diamonds denote decision, and parallelograms indicate input/output. Flowcharts make communication better among team members and enable easier documentation and understanding of a program's operation.

2 Basic Flowchart

A simple flowchart is a very useful yet straightforward tool employed for designing and analyzing the logic flow in a program. It shows the step-by-step visual outline of the series of operations. A simple flowchart usually starts with a "Start" symbol and ends with a "Stop" symbol. Between these two, actions, inputs/outputs, and decisions are represented with the respective shapes.

Take a simple example of finding out if a number is even or odd. The flowchart would start with the beginning symbol, followed by an input box for the number, followed by a decision box that determines if the number is divisible by 2. Based on the result, the program goes to either the "even" or "odd" output box before ending at the stop symbol. This type of graphical method allows for easier identification of mistakes or inefficiencies in the algorithm prior to any actual coding.

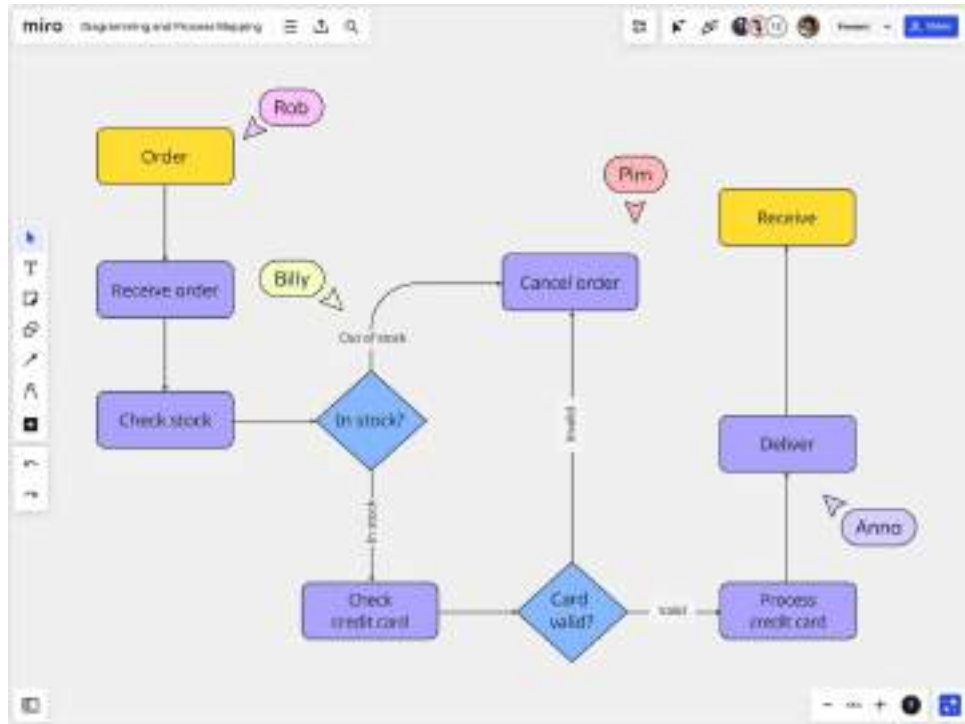


Figure 1: Basic flowchart representation [1]

3 Primitive Data Types and Input/Output

3.1 Primitive Data Types Defined

Primitive data types are the simplest types of data that may be utilized in programming. They are defined by the programming language and are used as building blocks for data manipulation. Typical primitive data types are:

- Integer: Stored whole numbers
- Float or Double: Stored decimal numbers
- Character: Stores a single character
- Boolean: Stores truth values, true or false

Knowing these types is important because choosing the right data type influences the efficiency and correctness of a program.

3.2 Input and Output

Input and output operations are critical in every interactive program. Input is how a program acquires data, usually from the user or other external source. Output is how a program presents or writes the results, often to the screen or a file. These processes allow for interaction between the user and the program.

3.3 Signed vs. Unsigned Data

A signed data type can hold both positive and negative values. An unsigned type, however, can only hold zero or positive values, which essentially doubles the upper limit of the values that it can hold. The choice between signed and unsigned is based on the needs of your program.

3.4 Constants and Variables

Variables are locations in memory that hold data values which can be modified during the running of a program. Constants are values that do not change. Using constants guarantees that certain values do not change throughout the program, which improves reliability and readability.

4 Operators

Operators are symbols that are used to perform operations on values and variables.

4.1 Arithmetic Operators

Arithmetic operators are addition, subtraction, multiplication, division, and modulus. They are used to do mathematical operations in programs and get used very often in expressions and formulae.

Arithmetic Operators		Let us assume X and Y are two variables.
Operator	Expression	Description
+	$X + Y$	To perform addition
-	$X - Y$	To perform subtraction
*	$X * Y$	To perform multiplication
/	X / Y	To perform division
%	$X \% Y$	To perform modulus

Figure 2: Flowchart structure and logic flow [3]

4.2 Increment and Decrement Operators

These operators are employed to increment or decrement the value of a variable by one. They are very helpful when a variable is being used as a counter, for example, in loops or iterations.

4.3 Relational Operators

Relational operators permit comparison of two values. They are applied in decision-making structures to test conditions, like testing whether one value is larger than another

or whether two values are equal.

4.4 Logical Operators

Logical operators are employed to test more than one condition. They are helpful in creating intricate decision-making logic where more than one criterion needs to be tested simultaneously.

4.5 Type Casting

Type casting is changing one data type to another, as changing a float into an integer to remove the decimal portion. This is most commonly necessary when uniting two different kinds of data in a calculation or comparison.

5 Bitwise Operators

Bitwise operators operate on data at the binary level, i.e., on the individual bits of a number. They include operations such as AND, OR, XOR, NOT, and bit shifts. Bitwise operations find application especially in low-level programming, device control, cryptography, and performance tuning.

6 Conditional/Selection Structures

6.1 If and If-Else

Use of the if statement is when a specific block of code must be executed only if a certain condition exists. If-else gives an alternative path if the condition does not exist. This is the basis of decision-making in computer programs.

6.2 Nested If-Else

Nested if-else statements enable one to test several levels of conditions. You can check for a condition within another condition, which comes in handy in larger decision trees.

6.3 Else-If Ladder

This construct assists in testing several different conditions consecutively. Every condition is tested successively until a true one is discovered.

6.4 Switch Statement

A switch statement serves as a neater way of avoiding multiple if-else statements in case of testing a single variable against different values. It makes code more readable and well-structured.

6.5 Ternary Operator

The ternary operator is an abbreviated way of coding simple if-else statements. It is concise and handy to use when assigning values under conditions.

7 Loops

Loops are employed to execute a block of code repeatedly.

7.1 While Loop

The while loop tests the condition before entering the loop body. It is handy when the number of iterations is unpredictable.

7.2 For Loop

The for loop is best suited when iterations are known. It has initialization, condition check, and increment/decrement all in one line, hence concise and convenient.

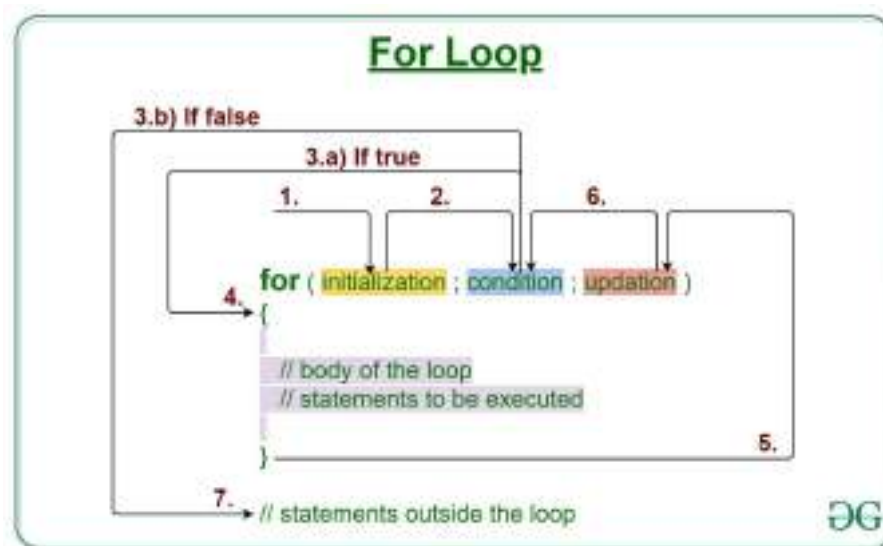


Figure 3: For loop illustration [2]

7.3 Do-While Loop

This loop ensures that the code within the loop is executed at least once because the condition is checked after executing the loop body.

7.4 Nested Loops

Nested loops are used when a loop is put inside another. It is helpful for processing multi-dimensional data like tables or matrices.

7.5 Break and Continue

The break statement comes out of the loop completely, and the continue statement bypasses the current iteration and goes on to the next iteration of the loop. Both are helpful in controlling the flow of loop execution.

8 Functions

Functions segment a program into manageable components. A function executes a specific operation and can be called many times in the program, which is encouraging modularity and avoiding redundancy.

8.1 Function Prototypes

Function prototypes notify the compiler of a function's return type and parameters prior to its definition. This guarantees correct function usage in the program.

8.2 Definition and Calling

Function definition includes the actual code doing a job. A function call is making it do its job at some point of the program.

8.3 Function Overloading

Function overloading enables several functions to have the same function name but with dissimilar parameter lists. It improves the flexibility and clarity of code.

8.4 Default Parameters

Default parameters allow us to specify predefined values for function arguments, such that it becomes unnecessary to provide all parameters on each call of the function.

9 Arrays

Arrays keep several values of the same data type under a single variable name. They prove helpful when there are collections of data like lists, sequences, or series to work with.

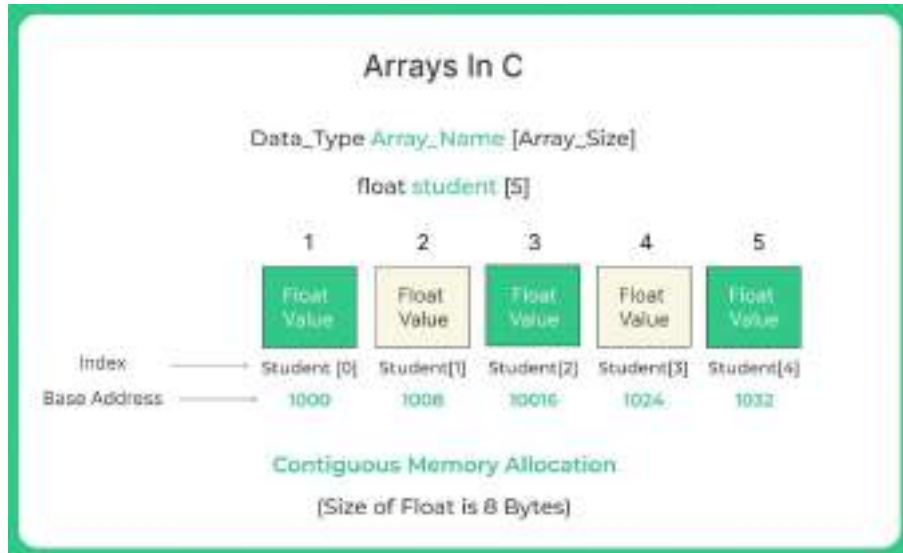


Figure 4: Array structure and representation [4]

9.1 Initializing Arrays

Arrays can be initialized when they are declared or in the latter part of the program. Initialization is important to guarantee that every array element contains a specified value before it is utilized.

9.2 Character Arrays

Character arrays, which are another term for C-style strings, contain character sequences. Although string objects are used in most modern programming, character arrays are essential for comprehending how text data is stored and manipulated at a lower level.

10 Conclusion

This course has also assisted me in establishing a solid and pragmatic base in programming. I've acquired the knowledge of writing functional yet simple programs and also made a clear understanding of data types, operators, decision structures, and loops. Through the use of flowcharts, I can design logic more effectively prior to coding, and through the application of functions and arrays, I have learned how to keep data organized and handled more efficiently.

Every subject progressed from the last, resulting in a holistic conception. The insight derived here has increased my logical thinking and enhanced my problem-solving strategy. These are not only valuable in coding but can be applied in mathematics, strategic development, and analytical work. Through further practice, I am increasingly confident that I can code clean, effective code and make positive contributions to coding projects.

References

- [1] Miro Blog. What is a flowchart?, 2024. <https://miro.com/flowchart/what-is-a-flowchart/>.
- [2] GeeksforGeeks. Loops in programming, 2024. <https://www.geeksforgeeks.org/loops-programming/>.
- [3] Pankaj Jha. Operators in c programming, 2024. <https://pankajj112223333.medium.com/operators-in-c-programming-i-bcb33d4adb5f>.
- [4] PrepInsta. Introduction to arrays in c, 2024. <https://prepinsta.com/c-program/introduction-to-arrays/>.